

7.5.3 Ricorrenza sui Costi

La scrittura della ricorrenza sui costi è il secondo passo per costruire un algoritmo di programmazione dinamica.

Definisco

$$l(i, j) = |LCS(X_i, Y_j)|$$

$$l(i, j) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 & (\text{caso 0}) \\ l(i-1, j-1) + 1 & \text{se } i, j > 0 \text{ e } x_i = x_j & (\text{caso 1}) \\ \max\{l(i, j-1), l(i-1, j)\} & \text{se } i, j > 0 \text{ e } x_i \neq x_j & (\text{caso 2}) \end{cases}$$

Alla fine ci interessa calcolare $l(m, n)$.

Per calcolare $l(i, j)$ mi possono servire tre valori:

$$L = \left[\begin{array}{cc} (i-1, j-1) & (i-1, j) \\ & \nwarrow \quad \uparrow \\ (i, j-1) & \leftarrow (i, j) \end{array} \right]$$

Scansione "row-major": riempio la tabella per righe, da sinistra a destra.

Informazione aggiuntiva per costruire la sequenza (vera e propria):

$$b(i, j) = \begin{cases} \nwarrow & \text{se } x_i = y_j \\ \leftarrow & \text{se } x_i \neq x_j \text{ e } \max = LCS(i, j-1) \\ \uparrow & \text{se } x_i \neq y_j \text{ e } \max = LCS(i-1, j) \end{cases}$$

PseudocodiceLCS(X, Y)

```
1   $m = X.length$ 
2   $n = Y.length$ 
3  for  $i = 0$  to  $m$ 
4       $L[i, 0] = 0$ 
5  for  $j = 0$  to  $n$ 
6       $L[0, j] = 0$ 
7  for  $i = 1$  to  $m$ 
8      for  $j = 1$  to  $n$ 
9          if  $x_i = y_j$ 
10              $L[i, j] = L[i - 1, j - 1] + 1$ 
11              $B[i, j] = '\nwarrow'$ 
12          else if  $L[i - 1, j] \geq L[i, j - 1]$ 
13              $L[i, j] = L[i - 1, j]$ 
14              $B[i, j] = '\uparrow'$ 
15          else
16              $L[i, j] = L[i, j - 1]$ 
17              $B[i, j] = '\leftarrow'$ 
18  return ( $L[m, n], B$ )
```

Complessità

$$T(m, n) = \Theta(m \cdot n)$$

$$\text{Caso } m = n \Rightarrow T(m, n) = \Theta(n^2)$$

Procedura per stampare la LCS:

PRINT-LCS(B, X, i, j)

```
1  if  $i = 0$  or  $j = 0$ 
2      return  $\varepsilon$ 
3  if  $B[i, j] = '\nwarrow'$ 
4      PRINT-LCS( $B, X, i - 1, j - 1$ )
5      PRINT( $x_i$ )
6  else if  $B[i, j] = '\leftarrow'$ 
7      PRINT-LCS( $B, X, i, j - 1$ )
8  else //  $B[i, j] = '\uparrow'$ 
9      PRINT-LCS( $B, X, i - 1, j$ )
```

Complessità $\Theta(m) = \Theta(|LCS|)$

Esercizio

$X = \langle b, d, c, d \rangle$

$Y = \langle a, b, c, b, d \rangle$

Restituisci $LCS(X, Y)$ e $|LCS(X, Y)|$

$$L = \begin{array}{c} \begin{array}{ccccc} & a & b & c & b & d \\ \begin{array}{c} b \\ d \\ c \\ d \end{array} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \mathbf{1} & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & \mathbf{2} & 2 \\ 0 & 0 & 1 & 2 & \mathbf{3} \end{bmatrix} \end{array} & B = \begin{array}{c} \begin{array}{ccccc} & a & b & c & b & d \\ \begin{array}{c} b \\ d \\ c \\ d \end{array} & \begin{bmatrix} \uparrow & \swarrow & \leftarrow & \swarrow & \leftarrow \\ \uparrow & \uparrow & \uparrow & \uparrow & \swarrow \\ \uparrow & \uparrow & \swarrow & \leftarrow & \uparrow \\ \uparrow & \uparrow & \uparrow & \uparrow & \swarrow \end{bmatrix} \end{array} \end{array}$$

$LCS(X, Y) = \langle b, c, d \rangle \quad |LCS(X, Y)| = 3$

Pseudocodice Memoizzato

INIT-LCS(X, Y)

```

1   $m = X.length$ 
2   $n = Y.length$ 
3  if ( $m = 0$ ) or ( $n = 0$ )
4      return 0
5  for  $i = 0$  to  $m$ 
6       $L[i, 0] = 0$ 
7  for  $j = 0$  to  $n$ 
8       $L[0, j] = 0$ 
9  for  $i = 1$  to  $m$ 
10     for  $i = 1$  to  $n$ 
11          $L[i, j] = -1$ 
12 return R-LCS( $X, Y, m, n$ )

```

R-LCS(X, Y, i, j)

```

1  if  $L[i, j] = -1$ 
2      if  $x_i = y_j$ 
3           $L[i, j] = \text{R-LCS}(X, Y, i - 1, j - 1)$ 
4      else if  $\text{R-LCS}(X, Y, i - 1, j) \geq \text{R-LCS}(X, Y, i, j - 1)$ 
5           $L[i, j] = L[i - 1, j]$ 
6      else
7           $L[i, j] = L[i, j - 1]$ 
8  return  $L[i, j]$ 

```

Complessità $O(m \cdot n)$

Osservazione Se $x_i = y_j$ sempre, invoco $\text{R-LCS}(X, Y, i-1, j-1)$ ma non invoco mai $\text{R-LCS}(X, Y, i-1, j)$ o $\text{R-LCS}(X, Y, i, j-1)$.

Ad esempio

$X = \langle a, a, b, b, c \rangle$

$Y = \langle b, b, c \rangle$

Albero delle chiamate:

$$\begin{array}{c}
 (5, 3) \\
 \left| \begin{array}{l} c = c \end{array} \right. \\
 (4, 2) \\
 \left| \begin{array}{l} b = b \end{array} \right. \\
 (3, 1) \\
 \left| \begin{array}{l} b = b \end{array} \right. \\
 (2, 0)
 \end{array}$$

In generale, se Y è suffisso di $n \leq m$ caratteri di X , la complessità di R-LCS nel caso migliore è:

$$T_{\text{R-LCS}}(m, n) = n$$

Inoltre,

$$\begin{aligned}
 \Omega_{\text{LCS}}(m + n) &\cong \Omega(n) \\
 O_{\text{LCS}, \text{R-LCS}}(m \cdot n) &\cong O(n^2)
 \end{aligned}$$

Spazio

$$S_{\text{LCS}}(m, n) = \Theta(m, n)$$

Tuttavia, posso migliorarlo a

$$\Theta(2n) = \Theta(n)$$

poichè mi bastano due righe della tabella in memoria ad ogni istante, quindi due vettori lunghi n .

Inoltre, se $m \ll n$, posso fare un'ulteriore ottimizzazione utilizzando la tecnica "column-major", cioè scansione per colonne, con due vettori lunghi m .

7.6 Longest Increasing Subsequence (LIS)

Def Dato un alfabeto Σ totalmente ordinato ($\forall a, b \in \Sigma \quad a < b \text{ o } a = b \text{ o } a > b$) e dato $X = \langle x_1, x_2, \dots, x_n \rangle$, si dice che $Z = \langle z_1, z_2, \dots, z_k \rangle$ è **sottosequenza crescente** di X ($Z = IS(X)$).

7.6.1 Problema di Ottimizzazione

Determinare la **più lunga sottosequenza crescente** di X ($Z = LIS(X)$)

Esempio

$$X = \langle 5, 2, 4, 3, 7, 8 \rangle$$

$$Z = LIS(X) = \langle 2, 3, 7, 8 \rangle$$

$$Z' = LIS(X) = \langle 2, 4, 7, 8 \rangle$$

Tentativo Data X , calcolo $LIS(X_i) \forall 0 \leq i \leq n$

$$Z = \langle z_1, z_2, \dots, z_k \rangle = LIS(X_i)$$

◦ caso fortunato: $z_k < X_{i+1}$

◦ caso sfortunato: $z_k \geq X_{i+1}$

Def $Z = \overline{LIS}(X_i)$ è la più lunga tra le $IS(X_i)$ con $Z = \langle z_1, z_2, \dots, z_k \rangle = \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle$ con $i_k = i$.

Esempio

$$X = \langle 8, 2, 5, 1, 3 \rangle$$

$$LIS(X_4) = \langle 2, 5 \rangle$$

$$\overline{LIS}(X_4) = \langle 1 \rangle$$

In generale, LIS e $\overline{LIS}$ sono problemi molto diversi.

Osservazione $|LIS(X)| = \max_{1 \leq i \leq n} \{|\overline{LIS}(X_i)|\}$

Dimostrazione

$$\circ |LIS(X)| \leq \max_{1 \leq i \leq n} \{|\overline{LIS}(X_i)|\}$$

$$Z = LIS(X) = \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle$$

$$Z = \overline{LIS}(X_{i_k})$$

$$\Rightarrow |Z| \leq \max_{1 \leq i \leq n} \{|\overline{LIS}(X_i)|\}$$

$$\circ |LIS(X)| \geq \max_{1 \leq i \leq n} \{|\overline{LIS}(X_i)|\}$$

Si dimostra analogamente al punto precedente.

7.6.2 Proprietà di Sottostruttura Ottima

$$1. \text{ caso base: } \overline{LIS}(X_1) = \langle x_1 \rangle (= LIS(X_1))$$

$$2. i > 1$$

$$(a) \forall j : 1 \leq j \leq i \quad x_j \geq x_i$$

$$\overline{LIS}(X_i) = \langle x_i \rangle (= LIS(X_i))$$

$$(b) \exists \bar{j} : 1 \leq \bar{j} \leq i, \quad x_{\bar{j}} < x_i$$

$$|\overline{LIS}(X_i)| \geq 2$$

$$\overline{LIS}(X_i) = \langle z_1, z_2, \dots, z_k \rangle = \langle Z_{k-1}, x_i \rangle \text{ con } Z_{k-1} : |Z_{k-1}| = \max_{1 \leq j < i} \{|\overline{LIS}(X_j)| : x_j < x_i\}$$

Dimostrazione

$$1. \text{ banale}$$

$$2. i > 1$$

$$(a) \forall j < i \quad x_j < x_i$$

$$\langle x_i \rangle = LIS(X_i) \text{ e chiaramente non può essere } |Z| > 1$$

Suppongo per assurdo che

$$|Z| = |\overline{LIS}(X_i)| > 1$$

$$\Rightarrow Z = \langle z_1, z_2, \dots, z_k \rangle, \quad k > 1$$

$$Z = \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle$$

$$i_k = i$$

$$\Rightarrow i_{k-1} < i_k = i$$

Assurdo perchè allora avrei

$$x_{i_{k-1}} \geq x_{i_k}$$

che contraddice l'ipotesi che Z è una sequenza crescente!

(b) Si dimostra analogamente al sottocaso precedente.

7.6.3 Ricorrenza sui Costi

Definisco

$$l(i) = |\overline{LIS}(X_i)|$$

$$l(i) = \begin{cases} 1 & \text{se } i = 1 \\ 1 + \max_{1 \leq j < i} \{l(j) : x_j < x_i\} & \text{se } i > 1 \end{cases}$$

Per costruire la sottosequenza

$$\overline{LIS}(X_i) = \begin{cases} \langle x_i \rangle & (1) \\ \langle \overline{LIS}(X_{\bar{j}}), x_i \rangle & \text{con } 1 \leq \bar{j} < i \quad (2) \end{cases}$$

l'informazione addizionale è:

- $prev(i) = \begin{cases} 0 & \text{nel caso (1)} \\ \bar{j} & \text{nel caso (2)} \end{cases}$
- $len = \max_{1 \leq i \leq n} \{l(i)\}$
- $end = i$ se $\overline{LIS}(X_i) = LIS(X)$
(mantiene l'indice dell'ultimo carattere della LIS)

Pseudocodice bottom-up

LIS(X)

```
1   $L[1] = 1$ 
2   $len = 1$ 
3   $end = 1$ 
4   $prev[1] = 0$ 
5  for  $i = 2$  to  $n$ 
6       $L[i] = 1$ 
7       $prev[i] = 0$ 
8      for  $j = 1$  to  $i - 1$ 
9          if  $x_j < x_i$ 
10             if  $L[i] < 1 + L[j]$ 
11                  $L[i] = 1 + L[j]$ 
12                  $prev[i] = j$ 
13     if  $len < L[i]$ 
14          $len = L[i]$ 
15          $end = i$ 
16 return ( $len, prev, end$ )
```

Procedura per stampare la LIS:

R-PRINT($X, prev, i$)

```
1  if  $prev[i] \neq 0$ 
2      R-PRINT( $X, prev, prev[i]$ )
3  PRINT( $x_i$ )
```

Complessità $\sum_{i=2}^n \sum_{j=1}^{i-1} 1 = \sum_{i=2}^n (i-1) = \frac{n(n-1)}{2} = \Theta(n^2)$

7.7 Completamento a Palindromo (CP)

Def Una stringa $Z = \langle z_1, z_2, \dots, z_m \rangle$ è **palindroma** se $z_{1+h} = z_{m-h} \quad \forall 0 \leq h \leq m-1$.

Problema Data $X = \langle x_1, x_2, \dots, x_n \rangle$, un **complemento palindromo** di X è una stringa $Z = CP(X) = \langle z_1, z_2, \dots, z_m \rangle$ con $m \geq n$ tale che:

- 1) Z è palindroma;
- 2) X è sottosequenza di Z

(cioè, Z è un palindromo che contiene X come sottosequenza).

Esempio

$$X = \langle a, c, b \rangle$$

$$Z = \langle a, c, b, b, c, a \rangle$$

$$Z' = \langle a, b, c, c, b, a \rangle$$

Osservazione $|X| = n \leq |Z| \leq 2n = 2|X|$

7.7.1 Problema di Ottimizzazione

Determinare $Z = CP(X)$ di lunghezza minima.

Spazio dei sottoproblemi: $X_{i...j}$ (cioè lo spazio delle sottostringhe di X).

Determinare un algoritmo bottom-up quadratico nella lunghezza di X .
 $\forall i, j : 1 \leq i \leq j \leq n$, determinare il minimo $Z = CP(X_{i...j})$.

7.7.2 Proprietà di Sottostruttura Ottima

Casi base:

1. $i = j$ (1 carattere)

$$X = \langle x_i \rangle$$

$$CP(X_{i...j}) = X_{i...j}$$

$$|CP(X_{i...j})| = |X_{i...j}|$$

2. $j = i + 1$ (2 caratteri)

$$X_{i...j} = \langle x_i, x_{i+1} \rangle$$

$$(a) \ x_i = x_{i+1}$$

$$CP(X_{i...i+1}) = X_{i...i+1}$$

$$|CP(X_{i...i+1})| = 2$$

$$(b) \ x_i \neq x_{i+1}$$

Un possibile $CP(X_{i...i+1})$ è $\langle x_i, x_{i+1}, x_i \rangle$

$$|CP(X_{i...i+1})| = 3$$

Caso generale:

$$X_{i...j} = \langle x_i, x_{i+1}, \dots, x_j \rangle$$

$$(a) \ x_i = x_j$$

$$Z = CP(X_{i...j})$$

$$z_1 = z_k = x_i (= x_j)$$

$$Z_{2...k-1} = CP(X_{i+1...j-1})$$

(b) $x_i \neq x_j$ Ci sono due possibili soluzioni:

$$\text{i. } z_1 = z_k = x_i \text{ e } Z_{2...k-1} = CP(X_{i+1...j})$$

$$\text{ii. } z_1 = z_k = x_j \text{ e } Z_{2...k-1} = CP(X_{i...j-1})$$

Dimostrazione

(b).i. Suppongo per assurdo che

$$z_1 = z_k \neq x_i$$

$\Rightarrow X_{i...j}$ è sottosequenza di

$Z_{2...k-1}$ che è palindroma e più corta di 2 rispetto a Z

Assurdo perchè Z è la più corta!

(b).ii. Si dimostra analogamente al sottocaso precedente.

7.7.3 Ricorrenza sulle Lunghezze

Definisco

$$l(i, j) = |CP(X_{i...j})|$$

$$l(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } j = i + 1 \text{ e } x_i = x_j \\ 3 & \text{se } j = i + 1 \text{ e } x_i \neq x_j \\ 2 + l(i + 1, j - 1) & \text{se } j > i + 1 \text{ e } x_i = x_j \\ 2 + \min\{l(i + 1, j), l(i, j - 1)\} & \text{se } j > i + 1 \text{ e } x_i \neq x_j \end{cases}$$

Per calcolare $l(i, j)$ mi possono servire tre valori:

$$L = \left[\begin{array}{ccc} & (i, j-1) & \leftarrow (i, j) \\ & \swarrow & \downarrow \\ (i+1, j-1) & & (i+1, j) \end{array} \right]$$

Riempio la tabella per diagonali, dall'alto verso il basso e da sinistra verso destra.

SPC(X)

```

1   $n = X.length$ 
2  for  $i = 1$  to  $n - 1$ 
3       $L[i, j] = 1$ 
4      if  $x_i = x_{i+1}$ 
5           $L[i, i + 1] = 2$ 
6      else
7           $L[i, i + 1] = 3$ 
8   $L[n, n] = 1$ 
9  for  $l = 3$  to  $n$  // scansione diagonale con  $l$  indice della diagonale
10     for  $i = 1$  to  $n - l + 1$ 
11          $j = i + l - 1$ 
12         if  $x_i = x_j$ 
13              $L[i, j] = 2 + L[i + 1, j - 1]$ 
14         else
15              $L[i, j] = 2 + \min\{L[i + 1, j], L[i, j - 1]\}$ 
16 return  $L[1, n]$ 
```

Complessità

$$\begin{aligned}
 & \sum_{i=1}^{n-1} 1 + \sum_{l=3}^n \sum_{i=1}^{n-l+1} 1 = \sum_{i=1}^{n-1} 1 + \sum_{l=3}^n (n-l+1) = \sum_{i=1}^{n-1} 1 + \sum_{i=2}^{n-1} (n-i) = \\
 & = \sum_{i=1}^{n-1} (n-i) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = \Theta(n^2)
 \end{aligned}$$